Experiment 7-Artificial Neural Network (ANN) without and backpropogation

Objective

Implementing artificial neural networks (ANNs) without and with backpropagation involves creating a model that can learn from data and make predictions.

Theory

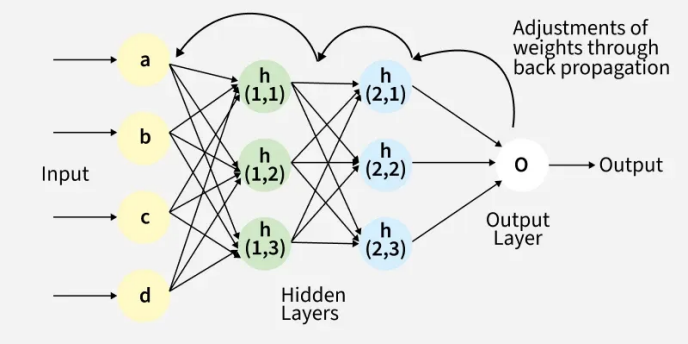

Figure 1: Structure of a Backpropogation Artificial Neural Network

Code

```
import numpy as np
import matplotlib.pyplot as plt

# Sigmoid activation
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Input data (AND gate - linearly separable)
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

# Desired output
y = np.array([[0],
              [0],
              [0],
              [1]])

np.random.seed(0)

# Network architecture
input_neurons = 2
hidden_neurons = 4
output_neurons = 1

# Initialize weights and biases
weights_input_hidden = np.random.rand(input_neurons, hidden_neurons)
bias_hidden = np.random.rand(hidden_neurons)

weights_hidden_output = np.random.rand(hidden_neurons, output_neurons)
bias_output = np.random.rand(output_neurons)

learning_rate = 0.1
epochs = 10000

# Training (Backpropagation)
for epoch in range(epochs):

    # Forward pass
```

```
    hidden_input = np.dot(X, weights_input_hidden) + bias_hidden
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, weights_hidden_output) + bias_output
    predicted_output = sigmoid(final_input)

    # Error
    error = y - predicted_output

    # Backward pass
    d_output = error * sigmoid_derivative(predicted_output)

    error_hidden = d_output.dot(weights_hidden_output.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    # Update weights and biases
    weights_hidden_output += hidden_output.T.dot(d_output) * learning_rate
    bias_output += np.sum(d_output, axis=0) * learning_rate

    weights_input_hidden += X.T.dot(d_hidden) * learning_rate
    bias_hidden += np.sum(d_hidden, axis=0) * learning_rate


# Testing
print("ANN Predictions:")
for i in range(len(X)):
    hidden = sigmoid(np.dot(X[i], weights_input_hidden) + bias_hidden)
    output = sigmoid(np.dot(hidden, weights_hidden_output) + bias_output)
    print(X[i], "->", np.round(output))


# Decision boundary plot
x1 = np.linspace(-0.2, 1.2, 200)
x2 = np.linspace(-0.2, 1.2, 200)
xx, yy = np.meshgrid(x1, x2)

grid = np.c_[xx.ravel(), yy.ravel()]

hidden_grid = sigmoid(np.dot(grid, weights_input_hidden) + bias_hidden)
output_grid = sigmoid(np.dot(hidden_grid, weights_hidden_output) + bias_output)

Z = (output_grid > 0.5).astype(int)
Z = Z.reshape(xx.shape)

plt.figure()
plt.contourf(xx, yy, Z, alpha=0.3)
```

```
plt.scatter(X[:, 0], X[:, 1], c=y.ravel(), edgecolors='black')
plt.xlabel("Input x1")
plt.ylabel("Input x2")
plt.title("Decision Boundary of Backpropagation ANN")
plt.show()
```

Output

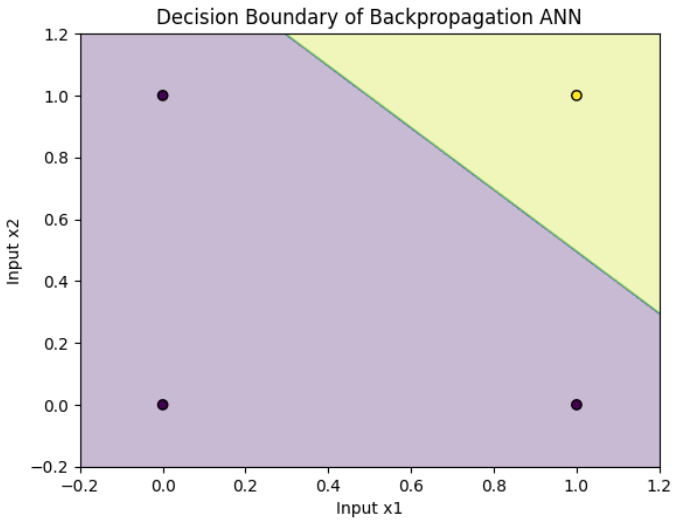

Figure 2: Decision boundary of the trained Artificial Neural Network using forward + backward pass

```
ANN Predictions:
[0 0] -> [0.]
[0 1] -> [0.]
[1 0] -> [0.]
[1 1] -> [1.]
```

Conclusion

Successfully implemented